

SQL Code

```
1  SELECT * FROM sales LIMIT 10;
2
3  -- Module 1: The Basics & Time Series
4  -- YoY sales and profit growth:
5
6  WITH yoy_growth AS(
7  SELECT
8      EXTRACT(YEAR FROM order_date) AS current_year,
9      SUM(sales) AS total_sales,
10     SUM(profit) AS total_profit,
11
12     LAG(SUM(sales)) OVER(ORDER BY EXTRACT(YEAR FROM order_date)) AS
prev_year_sales,
13     LAG(SUM(profit)) OVER(ORDER BY EXTRACT(YEAR FROM order_date)) AS
prev_year_profit
14 FROM sales
15 GROUP BY EXTRACT(YEAR FROM order_date)
16 )
17 SELECT *,
18     ROUND((total_sales - prev_year_sales) / prev_year_sales * 100,2) AS
yoy_sales_pct,
19     ROUND((total_profit - prev_year_profit) / prev_year_profit * 100,2) AS
yoy_profit_pct
20 FROM yoy_growth;
21
22
23 -- Seasonality Weekend effect total sales and total return:
24
25 SELECT
26     TO_CHAR(order_date,'Day') AS day_wise,
27     SUM(sales) AS total_sales,
28     SUM(CASE
29         WHEN returned ='Yes' THEN 1 ELSE 0 END) AS total_returns
30 FROM sales
31 GROUP BY TO_CHAR(order_date,'Day')
32 ORDER BY total_sales DESC;
33
34
35 -- Best quarter year total sales:
36
37 WITH ranked_sales AS(
38 SELECT
39     EXTRACT (YEAR FROM order_date) AS yearly,
40     EXTRACT (QUARTER FROM order_date) AS quarterly,
41     SUM(sales) AS total_sales
42 FROM sales
43 GROUP BY EXTRACT (QUARTER FROM order_date),EXTRACT (YEAR FROM order_date)
44 )
45 , sales_rn AS(
46 SELECT
```

```
47         *,
48         RANK() OVER(PARTITION BY yearly ORDER BY total_sales DESC) AS rn
49 FROM ranked_sales
50 )
51 SELECT
52     yearly,
53     quarterly,
54     total_sales
55 FROM sales_rn
56 WHERE rn = 1
57 ORDER BY yearly;
58
59 -- Module 2: Profit Bleeding (Where are we losing money?)
60 -- Where we are losing money [Discount trap]:
61
62 SELECT
63     category,
64     sub_category,
65     SUM(sales) AS total_sales,
66     SUM(profit) AS total_profit,
67     ROUND(AVG(discount)*100,2) AS avg_discount_pct
68 FROM sales
69 GROUP BY category,
70     sub_category
71 HAVING SUM(profit) < 0;
72
73
74 -- Return problems in regions with return pct:
75
76 SELECT
77     region,
78     SUM(sales) AS total_sales,
79     SUM(profit) AS total_profit,
80     COUNT(*) AS total_orders,
81     SUM(CASE WHEN returned = 'Yes' THEN 1 ELSE 0 END) AS total_returns,
82     ROUND(SUM(CASE WHEN returned = 'Yes' THEN 1 ELSE 0 END)::numeric * 100.0 /
83     COUNT(*),2) AS return_pct
84 FROM sales
85 GROUP BY region
86 ORDER BY return_pct DESC;
87
88 -- Top 10 loss making customers:
89
90 SELECT
91     customer_id,
92     customer_name,
93     SUM(sales) AS total_sales,
94     SUM(profit) AS total_loss
95 FROM sales
96 GROUP BY customer_id,customer_name
97 HAVING SUM(profit) <0
98 ORDER BY total_loss
99 LIMIT 10;
```

```

100
101 --Module 3: Advanced Supply Chain & Logistics
102 -- Shipping Delay Analysis:
103
104 SELECT
105     ship_mode,
106     --AVG(AGE(ship_date, order_date)) AS delay_days
107     ROUND((AVG(ship_date - order_date)),2) AS delay_days
108 FROM sales
109 GROUP BY ship_mode
110 ORDER BY delay_days;
111
112
113 -- Late delivery Impact Compare on time vs late returns:
114
115 WITH shipping_data AS(
116     SELECT
117         order_id,
118         returned,
119         ship_date - order_date AS deliver_days
120     FROM sales
121 )
122 SELECT
123     CASE
124         WHEN deliver_days > 3 THEN 'Late' ELSE 'On_time' END AS
125     delivery_status,
126     SUM( CASE
127         WHEN returned = 'Yes' THEN 1 ELSE 0 END ) AS return_orders,
128     COUNT(DISTINCT order_id) AS total_orders,
129     ROUND(COUNT(DISTINCT CASE
130         WHEN returned = 'Yes' THEN 1 ELSE 0 END ) * 100.0 /
131     COUNT(DISTINCT order_id) * 100 ,2) AS return_pct
132 FROM shipping_data
133 GROUP BY CASE
134     WHEN deliver_days > 3 THEN 'Late' ELSE 'On_time' END
135
136
137 -- Salesperson performance:
138
139 SELECT
140     retail_sales_people,
141     SUM(sales) AS total_sales,
142     SUM(profit) AS total_profit,
143     ROUND(SUM(profit) / SUM(sales) * 100,2) AS profit_margin
144 FROM sales
145 GROUP BY retail_sales_people
146
147 -- (Window Functions & CTEs)
148 -- Churn customers : placed order in 2015 & 2016 but never placed in 2017:
149
150 WITH customer_details AS(
151     SELECT
152         customer_id,

```

```
152         EXTRACT(YEAR FROM order_date) AS years
153     FROM sales
154 )
155 SELECT DISTINCT customer_id
156 FROM customer_details
157 GROUP BY customer_id
158 HAVING COUNT(DISTINCT CASE WHEN years IN (2015, 2016) THEN years END) = 2
159     AND
160     COUNT(DISTINCT CASE WHEN years = 2017 THEN years END) = 0;
161
162
163 -- 30 days Moving average:
164
165 WITH daily_sales AS(
166 SELECT
167     order_date,
168     SUM(sales) AS daily_sales
169 FROM sales
170 GROUP BY order_date
171 )
172 SELECT
173     order_date,
174     daily_sales,
175     AVG(daily_sales) OVER(ORDER BY order_date ROWS BETWEEN 29 PRECEDING AND
176 CURRENT ROW) AS moving_avg
177 FROM daily_sales
178
179 -- Customer Segmentation:
180
181 SELECT
182     customer_id,
183     customer_name,
184     COUNT( customer_id) AS total_orders,
185     SUM(sales) AS total_sales,
186     CASE
187         WHEN COUNT( customer_id)= 1 THEN 'One_time_buyer'
188         WHEN SUM(sales) > 5000 THEN 'VIP'
189         ELSE 'Regular'
190     END AS segments
191 FROM sales
192 GROUP BY customer_id,
193     customer_name
194 ORDER BY total_sales DESC;
195
196
197 -- MoM growth with pct
198
199 WITH month_growth AS(
200 SELECT
201     DATE_TRUNC('month', order_date)::DATE AS month_date,
202     EXTRACT(MONTH FROM order_date) AS month_number,
203     TRIM(TO_CHAR(order_date, 'Mon')) AS month_name,
204     SUM(sales) AS total_sales
```

```
205 FROM sales
206 GROUP BY DATE_TRUNC('month', order_date)::DATE,
207          EXTRACT(MONTH FROM order_date),
208          TRIM(TO_CHAR(order_date, 'Mon'))
209 )
210 SELECT
211     *,
212     LAG(total_sales) OVER(ORDER BY month_date) AS previous_month_sales,
213     ROUND((total_sales - LAG(total_sales) OVER(ORDER BY month_date)) *100 /
214     LAG(total_sales) OVER(ORDER BY month_date),2)as pct
215 FROM month_growth
216
217 -- Most profitable route:
218
219 SELECT
220     state,
221     city,
222     COUNT(order_id) as total_orders,
223     SUM(profit) as total_profit,
224     ROUND(SUM(profit) / COUNT(order_id),2) AS per_order_profit
225 FROM sales
226 GROUP BY state,
227          city
228 HAVING COUNT(order_id) >=10
229 ORDER BY total_profit DESC;
230
231
232
233
234
```